

MOHAMMED Fahad

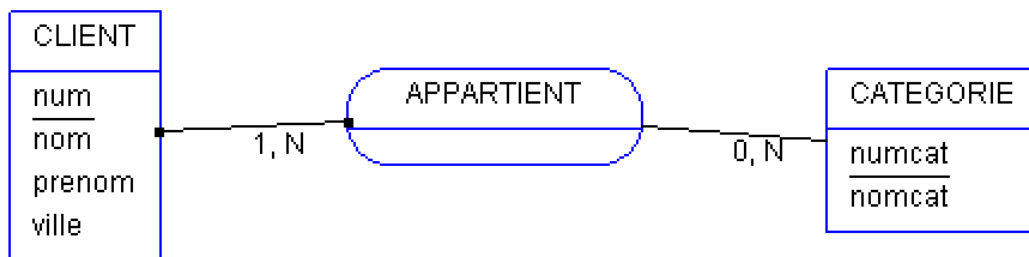
SIO1

20/01/2026

Objectif : Créer et manipuler une base de données relationnelle pour gérer les clients et leurs catégories chez Leclerc.

Outils utilisés : XAMPP (Apache + MySQL) avec phpMyAdmin.

Question 1 : Faire le MRD de ce schéma



Un client appartient à une seule catégorie (1,1), une catégorie peut concerner plusieurs clients (0,N).

Question 2 : sur un outil de base de données, créer la structure de la BDD

Pour créer la structure de la base de données, nous avons utilisé XAMPP et son outil phpMyAdmin. Nous avons d'abord démarré les services Apache et MySQL via le XAMPP. Ensuite, nous avons accédé à phpMyAdmin (<http://localhost/phpmyadmin>) et créé une nouvelle base de données nommée "tp_leclerc".

ÉTAPE 1 : CRÉATION DE LA BASE DE DONNÉES "leclerc"

Bases de données

Création d'une base de données

leclerc utf8mb4_general_ci Créer

ÉTAPE 2 : CRÉATION DE LA TABLE "categorie"

 **Créer une nouvelle table**

Nom de table

Nombre de colonnes

categorie

2

 **Chargement en cours...**

Une table est une structure qui organise les données en colonnes et lignes. La table "categorie" contient deux colonnes : numcat (un nombre entier qui sert d'identifiant unique) et nomcat (le nom textuel de la catégorie). La clé primaire sur numcat garantit qu'aucune catégorie n'a le même identifiants

Nom	Type	Taille/Valeurs*	Valeur par défaut	Interclassement	Attributs	Null	Index	A_I
numcat	INT		Aucun(e)			☐	PRIMARY	☐
Choisir à partir des colonnes centrales								
nomcat	VARCHAR		Aucun(e)			☐	---	☐
Choisir à partir des colonnes centrales								

ÉTAPE 3 : INSÉRER LES DONNÉES DANS CATEGORIE

```
1 INSERT INTO CATEGORIE (numcat, nomcat) VALUES
2 (1, 'non renseigné'),
3 (2, 'ouvrier'),
4 (3, 'cadre'),
5 (4, 'sans-emploi'),
6 (5, 'autres');
```

Maintenant qu'on a créé la table CATEGORIE, on l'a remplie avec 5 types de clients. On a mis un numéro pour chaque type : 1 pour "non renseigné" (quand on ne sait pas), 2 pour "ouvrier", 3 pour "cadre", 4 pour "sans-emploi", et 5 pour "autres". Ces catégories permettent à Leclerc de classer ses clients selon leur situation professionnelle. C'est utile parce que les cadres n'achètent pas forcément les mêmes choses que les ouvriers ou les sans-emploi. On a fait tout ça en une seule commande pour aller plus vite. À la fin, l'ordinateur nous a dit "5 lignes insérées", ce qui veut dire que tout s'est bien passé.

ÉTAPE 5 : INSERTION DES CLIENTS

```
INSERT INTO client (num, nom, prenom, ville, Categorie) VALUES
(1, 'Qausi', 'Nathan', 'Paris', 1),
(2, 'Vimif', 'Lucas', 'Paris', 3),
(3, 'Gulya', 'Enzo', 'Courbevoie', 3),
(4, 'Vetea', 'Léo', 'Nanterre', 4),
(5, 'Lossu', 'Louis', 'Paris', 4),
(6, 'Ebote', 'Emma', 'Paris', 2),
(7, 'Talam', 'Léa', 'Courbevoie', 3),
(8, 'Julyn', 'Chloé', 'Paris', 2),
(9, 'Voduk', 'Manon', 'Paris', 5),
(10, 'Tuuji', 'Inès', 'Nanterre', 2);
```

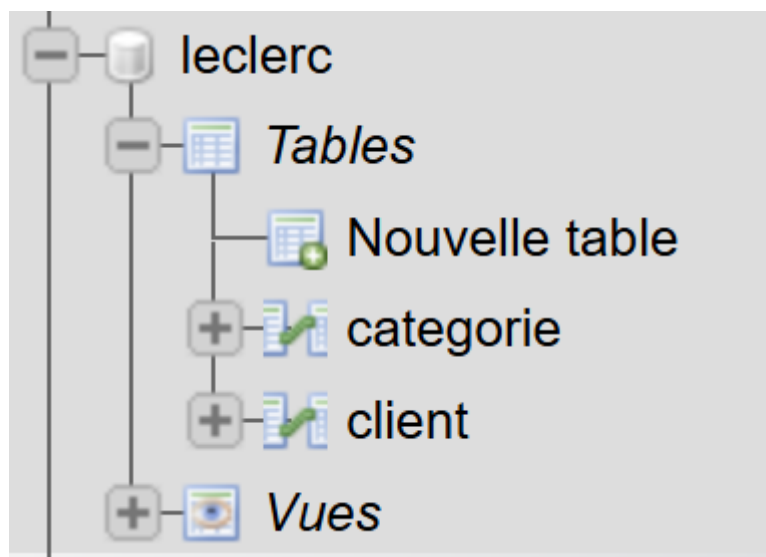
On a ajouté 10 clients dans la base de données avec toutes leurs informations : leur numéro, leur nom, leur prénom, leur ville et leur catégorie. Par exemple, le client numéro 1 s'appelle Nathan Qausi, il habite à Paris et il est dans la catégorie 1 (non renseigné). Le client numéro 2 s'appelle Lucas Vimif, il habite aussi à Paris mais il est dans la catégorie 3 (cadre). Quand on regarde tous les clients, on voit que 6 personnes habitent à Paris, 2 à Courbevoie, et 2 à Nanterre. On voit aussi qu'il y a 3 cadres et 3 ouvriers. Toutes ces informations vont servir à Leclerc pour mieux connaître ses clients. L'ordinateur a confirmé "10 lignes insérées", donc tout est bien enregistré.

QUESTION 4 : CRÉER UNE VUE POUR AFFICHER LES NOMS DES CATÉGORIES

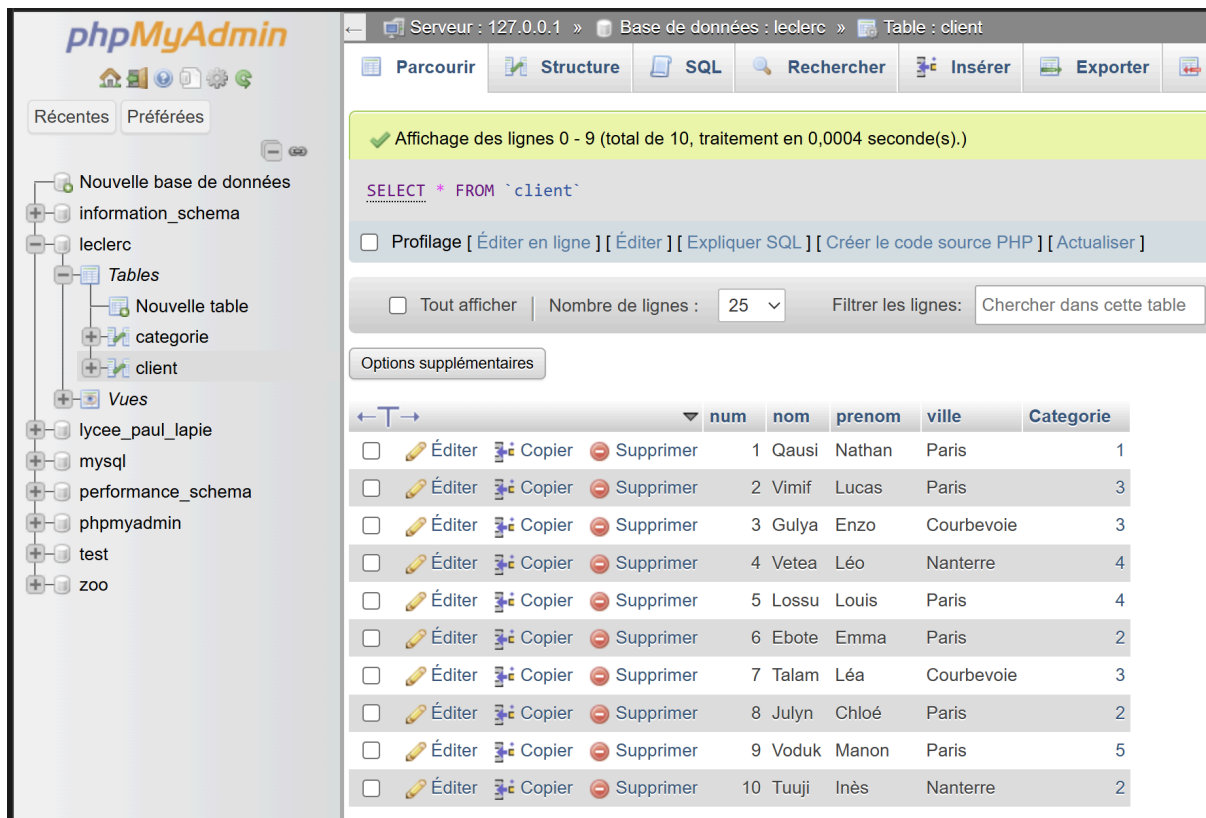
ÉTAPE 5 : CRÉATION DE LA VUE "vue_categories"

```
CREATE VIEW vue_categories AS  
SELECT nomcat FROM CATEGORIE;
```

On a créé quelque chose qu'on appelle une "vue". Une vue, c'est comme un raccourci. Au lieu de taper une longue commande à chaque fois, on crée un raccourci qu'on peut réutiliser facilement. Cette vue s'appelle `vue_categories` et elle montre juste les noms des catégories (ouvrier, cadre, etc.) sans les numéros. C'est plus simple à lire pour les humains. Par exemple, au lieu de voir "3", on verra directement "cadre". Pour Leclerc, c'est pratique quand on veut faire une liste pour les employés ou créer un menu déroulant sur un site web. La vue ne stocke pas les données, elle va juste les chercher dans la table `CATEGORIE` à chaque fois qu'on la regarde. C'est comme mettre un signet dans un livre pour retrouver rapidement une page.



QUESTION 5 : AFFICHER TOUTE LA LISTE DES CLIENTS



The screenshot shows the phpMyAdmin interface. On the left, the database structure is visible, including the 'leclerc' database and its tables: 'categorie' and 'client'. The 'client' table is selected, and the SQL query 'SELECT * FROM `client`' is entered in the query box. The table structure is defined as:

num	nom	prenom	ville	Categorie
1	Qausi	Nathan	Paris	1
2	Vimif	Lucas	Paris	3
3	Gulya	Enzo	Courbevoie	3
4	Vetea	Léo	Nanterre	4
5	Lossu	Louis	Paris	4
6	Ebote	Emma	Paris	2
7	Talam	Léa	Courbevoie	3
8	Julyn	Chloé	Paris	2
9	Voduk	Manon	Paris	5
10	Tuuji	Inès	Nanterre	2

On a affiché tous les clients avec toutes leurs informations. On voit les 10 clients avec leur numéro, leur nom, leur prénom, leur ville et leur catégorie. C'est une commande de base qu'on utilise souvent pour vérifier que les informations sont bien enregistrées. Pour Leclerc, cette commande peut servir à créer des listes complètes de clients, par exemple pour envoyer des catalogues à tout le monde. Dans la vraie vie, avec des milliers de clients, on utiliserait plutôt des filtres pour ne voir qu'une partie des clients, sinon la liste serait beaucoup trop longue. Mais ici avec seulement 10 clients, c'est parfait pour tout voir d'un coup.

QUESTION 6 : CRÉER UNE VUE POUR AFFICHER LES NOMS DES VILLES

```
1 CREATE VIEW vue_villes AS  
2 SELECT DISTINCT ville FROM client;
```

On a créé une deuxième vue qui montre la liste des villes où habitent les clients. Le mot `DISTINCT` est important : il enlève les doublons. Sans ce mot, on verrait "Paris" 6 fois parce qu'il y a 6 clients à Paris. Avec `DISTINCT`, on ne voit "Paris" qu'une seule fois. Le résultat montre 3 villes : Paris, Courbevoie et Nanterre. Pour Leclerc, cette vue est utile pour savoir où sont ses clients. Par exemple, si Leclerc veut distribuer des prospectus dans une ville, il peut d'abord vérifier s'il a des clients là-bas. C'est aussi utile pour décider où ouvrir un nouveau magasin. Si beaucoup de clients habitent dans une ville où il n'y a pas encore de magasin, ça pourrait être une bonne idée d'en ouvrir un là-bas.

QUESTION 7 : CRÉER UNE VUE POUR AFFICHER LES VILLES ET LES CATÉGORIES

```
1 CREATE VIEW vue_villes_categories AS  
2 SELECT DISTINCT client.ville, CATEGORIE.nomcat  
3 FROM client, CATEGORIE  
4 WHERE client.Categorie = CATEGORIE.numcat;
```

Cette troisième vue est un peu plus compliquée parce qu'elle mélange les informations de deux tables. On prend les villes de la table `client` et les noms de catégories de la table `CATEGORIE`, et on les met ensemble. Le résultat montre pour chaque ville quels types de clients y habitent. Par exemple, on peut voir qu'à Paris il y a les 5 types de clients, mais qu'à Courbevoie il y a seulement des cadres et des ouvriers. Pour Leclerc, c'est super utile pour savoir qui habite où. Si une ville a beaucoup de cadres, Leclerc peut y proposer plus de produits chers et de qualité. Si une ville a surtout des ouvriers ou des sans-emploi, Leclerc peut faire plus de

promotions sur les produits de base. C'est comme avoir une carte qui montre les différents types de clients dans chaque quartier.z

QUESTION 8 : TROUVER LA VILLE AVEC TOUTES LES CATÉGORIES

```
1 SELECT ville
2 FROM client
3 GROUP BY ville
4 HAVING COUNT(DISTINCT Categorie) = 5;
```

Cette commande cherche quelle ville a les 5 types de clients. C'est un peu comme compter combien de couleurs différentes il y a dans chaque boîte de bonbons. GROUP BY ville met tous les clients de chaque ville ensemble. Ensuite, COUNT(DISTINCT Categorie) compte combien de catégories différentes il y a dans chaque ville. Enfin, HAVING = 5 garde seulement les villes qui ont exactement 5 catégories. Le résultat montre que Paris est la seule ville avec les 5 types de clients. C'est important pour Leclerc parce que ça veut dire que Paris a tous les types de clients : des riches, des pauvres, des employés, des cadres, etc. C'est la ville la plus diverse. Leclerc peut donc y tester toutes sortes de produits et de promotions pour voir ce qui marche avant de le faire ailleurs.



ville

☐  Éditer  Copier  Supprimer Paris

QUESTION 9 : CRÉER UNE VUE POUR AFFICHER NOM, PRÉNOM ET CATÉGORIE

```
CREATE VIEW vue_clients_categories AS
SELECT client.nom, client.prenom, CATEGORIE.nomcat
FROM client, CATEGORIE
WHERE client.Categorie = CATEGORIE.numcat;
```

Cette vue montre chaque client avec son nom, son prénom et le nom de sa catégorie (pas le numéro). On mélange les deux tables pour remplacer les numéros par des mots. Au lieu de voir "Lucas Vimif - 3", on voit "Lucas Vimif - cadre", ce qui est beaucoup plus clair. C'est comme traduire des codes en langage normal. Pour les employés de Leclerc qui ne connaissent pas les numéros par cœur, c'est beaucoup plus facile à comprendre. Cette vue peut servir à faire des listes pour les campagnes publicitaires. Par exemple, si Leclerc veut envoyer une offre spéciale aux cadres, il suffit de regarder cette vue et de prendre tous ceux qui ont "cadre" marqué. C'est simple et rapide, et ça évite les erreurs.

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0009 seconde(s).)

```
CREATE VIEW vue_clients_categories AS SELECT client.nom, client.prenom, CATEGORIE.nomcat FROM client, CATEGORIE WHERE client.Categorie = CATEGORIE.numcat;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

QUESTION 10 : REPRENDRE LA QUESTION 9 ET TRIER PAR NOM

```
1 CREATE VIEW vue_clients_categories_trie AS
2 SELECT client.nom, client.prenom, CATEGORIE.nomcat
3 FROM client, CATEGORIE
4 WHERE client.Categorie = CATEGORIE.numcat
5 ORDER BY client.nom ASC;
```

Cette vue est exactement la même que la précédente, mais rangée par ordre alphabétique des noms de famille. C'est comme ranger des livres dans une bibliothèque de A à Z. ORDER BY client.nom ASC veut dire "range par nom, du A vers le Z" (ASC = ascendant, qui monte). C'est pratique quand on cherche quelqu'un dans une liste ou quand on veut imprimer un document bien organisé. Pour Leclerc, cette liste rangée peut servir pour les réunions, pour partager le travail entre plusieurs employés (l'un s'occupe de A à M, l'autre de N à Z), ou pour créer des rapports propres à montrer au directeur. Quand les noms sont dans l'ordre alphabétique, c'est beaucoup plus facile de trouver quelqu'un rapidement.

QUESTION 11 : AFFICHER LES CLIENTS HABITANT À COURBEVOIE

```
1 SELECT * FROM client WHERE ville = 'Courbevoie';
```

↩ T →						num	nom	prenom	ville	Categorie
✎ Éditer 📋 Copier 🗑 Supprimer	3	Gulya	Enzo	Courbevoie	3					
✎ Éditer 📋 Copier 🗑 Supprimer	7	Talam	Léa	Courbevoie	3					

Cette commande montre seulement les clients qui habitent à Courbevoie. Le WHERE ville = 'Courbevoie' est comme un filtre qui ne laisse passer que les clients de cette ville. C'est comme quand on cherche dans un annuaire téléphonique juste les gens d'une certaine ville. Le résultat montre 2 clients : Gulya Enzo et Talam Léa, tous les deux cadres. Pour Leclerc, c'est utile pour cibler une ville précise. Par exemple, si Leclerc veut ouvrir un nouveau magasin à Courbevoie, il peut voir combien de clients il a déjà là-bas. Avec seulement 2 clients, ça peut vouloir dire deux choses : soit il n'y a pas beaucoup de gens intéressés par Leclerc dans cette ville, soit il y a un gros potentiel pour attirer plus de clients avec de la publicité locale.

QUESTION 12 : AFFICHER LES CLIENTS DE CATÉGORIE "CADRE"

```
1 SELECT client.*
2 FROM client, CATEGORIE
3 WHERE client.Categorie = CATEGORIE.numcat
4 AND CATEGORIE.nomcat = 'cadre';
```

↩ T →						num	nom	prenom	ville	Categorie
✎ Éditer 📋 Copier 🗑 Supprimer	2	Vimif	Lucas	Paris	3					
✎ Éditer 📋 Copier 🗑 Supprimer	3	Gulya	Enzo	Courbevoie	3					
✎ Éditer 📋 Copier 🗑 Supprimer	7	Talam	Léa	Courbevoie	3					

Cette commande montre tous les clients qui sont des cadres. On mélange les deux tables et on garde seulement ceux qui ont "cadre" comme catégorie. Le résultat montre 3 clients cadres : Vimif Lucas, Gulya Enzo et Talam Léa. Ça fait 30% de tous les clients (3 sur 10). Pour Leclerc, c'est très utile de savoir combien de cadres il a comme clients. Les cadres ont généralement plus d'argent à dépenser, donc Leclerc peut leur proposer des produits plus chers et de meilleure qualité. Par exemple, Leclerc pourrait leur envoyer une invitation à une dégustation de vins fins, leur proposer un service de livraison à domicile haut de gamme, ou leur offrir une carte de fidélité avec plus d'avantages. On peut facilement changer "cadre" par "ouvrier" ou "sans-emploi" pour voir les autres groupes de clients.

QUESTION 13 : AFFICHER LE NOMBRE DE CLIENTS À COURBEVOIE

```
1 SELECT COUNT(*) AS nombre_clients
2 FROM client
3 WHERE ville = 'Courbevoie';
```

nombre_clients
2

Cette commande compte combien de clients habitent à Courbevoie. Au lieu de montrer tous les clients et de les compter à la main, l'ordinateur fait le calcul automatiquement avec COUNT(*). Le résultat est 2 clients. Le AS nombre_clients donne un nom plus clair au résultat : au lieu de voir juste un chiffre, on voit "nombre_clients: 2", ce qui est plus compréhensible. Pour Leclerc, compter vite comme ça évite de se tromper et fait gagner du temps. Avec des milliers de clients dans la vraie vie, ce serait impossible de compter à la main. Leclerc peut utiliser ces comptages pour décider si ça vaut le coup de faire de la publicité dans une ville. Avec seulement 2 clients à Courbevoie, peut-être qu'une grosse campagne publicitaire serait trop chère par rapport au nombre de clients. Ou au contraire, peut-être que c'est le moment d'investir pour en attirer plus.

QUESTION 14 : AFFICHER LE NOMBRE DE CLIENTS "CADRE"

```
1 SELECT COUNT(*) AS nombre_cadres
2 FROM client, CATEGORIE
3 WHERE client.Categorie = CATEGORIE.numcat
4 AND CATEGORIE.nomcat = 'cadre';
```

nombre_cadres

3

Cette dernière commande compte combien de clients sont des cadres. On mélange les deux tables et on compte seulement ceux qui sont dans la catégorie "cadre". Le résultat est 3 cadres, ce qui représente 30% de tous les clients. Savoir combien il y a de clients dans chaque catégorie aide Leclerc à mieux organiser son magasin. Si beaucoup de clients sont des cadres, Leclerc mettra plus de produits chers dans les rayons. Si beaucoup de clients ont peu d'argent, Leclerc fera plus de promotions. Ces chiffres servent aussi à mesurer si les campagnes publicitaires marchent bien. Par exemple, après avoir fait de la pub ciblée sur les cadres, Leclerc peut recompter pour voir s'il y a plus de cadres qu'avant. On peut facilement changer "cadre" par n'importe quelle autre catégorie pour avoir les chiffres de tous les groupes.

CONCLUSION

Dans ce travail, on a créé une base de données pour le magasin Leclerc. On a fait deux tableaux : un pour les types de clients (ouvrier, cadre, etc.) et un autre pour les clients avec leur nom, prénom et ville.

On a relié ces deux tableaux ensemble pour que tout soit bien organisé. Ça évite de faire des erreurs quand on ajoute des informations.

Ensuite, on a créé des "vues" qui sont comme des raccourcis pour voir rapidement certaines informations sans retaper les mêmes commandes à chaque fois.

On a aussi fait des recherches dans la base de données. On a trouvé que Paris est la seule ville avec tous les types de clients. On a compté combien de clients habitent dans chaque ville et combien sont des cadres, des ouvriers, etc.

Pour Leclerc, cette base de données est très utile. Elle permet de savoir qui sont les clients, où ils habitent et quel est leur métier. Comme ça, Leclerc peut envoyer les bonnes publicités aux bonnes personnes. Par exemple, proposer des produits chers aux cadres et faire plus de promotions pour les clients qui ont moins d'argent.

Grâce à ce travail, on a appris à créer des tableaux, les relier entre eux, ajouter des informations, faire des recherches et compter des résultats. Toutes ces techniques peuvent être utilisées dans la vraie vie avec des milliers de clients, pas seulement 10 comme dans notre exercice.